Arquitetura de Solução e Decisões Técnicas — SaborDaTerraAI

Especificação consolidada de decisões arquiteturais (ADRs), evidências de benchmark e contorno técnico para modelagem de System Design.

PROJETO	Assistente Inteligente — Cooperativa Sabor da Terra	**VERSÃO / DATA**	v2.1 (Baseline Consolidado) • 14/11/2026
RETRATO DOS DADOS	30/10/2026 (Dataset Operacional e Documental)	**MARCOS OBRIGATÓRIOS**	Arquitetura: **19/11/2026** • Demo: **20–21/11/2026**
ALVO DE DEPLOY	Google Cloud Run + Cloud SQL (PostgreSQL)	**STATUS**	Baseline Aprovado para Desenho de System Design

1. Contexto, Escopo e Requisitos Não-Funcionais

O **SaborDaTerraAI** é um assistente conversacional interno para a equipe de operações da Cooperativa Sabor da Terra (180 produtores em 12 municípios de Pernambuco, com volume diário de 60 a 100 pedidos), projetado para atender três frentes operacionais:

1. **Interpretação e Triagem de Pedidos:** Extração estruturada e classificação de pedidos em linguagem natural coloquial, com abreviações e informações implícitas.
2. **Consulta Inteligente à Base Documental e Operacional:** Respostas fundamentadas sobre certificações orgânicas, laudos sanitários, inspeção ADAGRO, contratos, fichas técnicas, saldos e rastreabilidade de lotes com citação obrigatória da fonte.
3. **Comunicação com Produtores:** Geração de resumos financeiros/operacionais e rascunhos de notificação para alocação de pedidos.

Restrições Críticas do Domínio e do Edital

- **Otimização Financeira:** Minimizar desembolso aproveitando créditos ativos no **Google AI Studio** e camadas gratuitas/ultrabarata, documentando projeções transparentes de custo no Google Cloud.
- **SLA de Latência:** Máximo de **10 segundos** ponta a ponta por solicitação individual (incluindo validações, consultas SQL, busca vetorial e eventual fallback).
- **Segurança Factual (Zero Alucinação):** Proibido inferir estoque, vigência de certificação orgânica, saldo financeiro ou liberação sanitária sem dado verificável em fonte oficial.
- **Temporalidade Controlada:** Regras sensíveis a datas operam sobre uma data de referência configurável (padrão: `30/10/2026`), evitando dependência de `today()`.
- **Anomalias Conhecidas nos Dados:** O acervo contém inconsistências propositais que exigem bloqueio determinístico — como certificação vencida em cadastro ainda marcado como "orgânico" (`P008`), restrição sanitária ativa para o frango (`P017`), unidades misturadas em `quantidade_kg` (unidades e dúzias) e datas contraditórias.
- **Diretriz Google Cloud:** A arquitetura deve evidenciar aplicação prática do ecossistema Google Cloud conforme escopo da parceria UPE / Sauter.

2. Decisões Arquiteturais Consolidadas (ADRs)

A tabela abaixo estabelece as decisões técnicas já definidas pela equipe como fundação obrigatória para a etapa de System Design:

Domínio	Decisão Consolidada	Justificativa Técnica e Econômica
Topologia do Agente	**Agente Único com Ferramentas (Single-Agent + Tools)** via **PydanticAI**	Menor latência, menor consumo de tokens e maior previsibilidade do que sistemas multiagentes. A separação de responsabilidades ocorre via ferramentas tipadas e regras determinísticas fora do prompt.
Interface e Empacotamento	**Módulo Único (Streamlit + Agente Python) Dockerizado**	Acelera a entrega dentro do prazo (19/11), aplica trava de turno por sessão para evitar concorrência no chat e simplifica o empacotamento em contêiner único.
Alvo de Implantação	**Google Cloud Run + Cloud SQL (PostgreSQL)** *(Docker Compose para dev local)*	Atende ao requisito de Google Cloud com escalabilidade automática (custo zero quando ocioso no Cloud Run) e portabilidade idêntica entre ambiente local e nuvem.
Persistência Única	**Instância Única PostgreSQL + Extensão `pgvector`** *(sem Redis ou brokers)*	Elimina custos e complexidade de múltiplos bancos ou filas externas. Uma única instância abriga tabelas operacionais (CSV/JSON), vetores (TXT), cache leve e logs de auditoria.
LLM Principal	**Gemini API Direta (`gemini-3.5-flash-lite`)**	Consome diretamente os créditos do Google AI Studio com baixa latência e suporte nativo a *Structured Outputs* e *Tool Calling*. *(Pendente validação direta nos 21 casos)*.
Fallback de Modelos	**OpenRouter (`Qwen3 32B` → `Mistral Small 3.2`)**	Redundância entre provedores independentes para falhas de disponibilidade, *rate limit* (HTTP 429) ou erro de schema na API principal.
Modelo de Embedding	**`gemini-embedding-001` (Google AI Studio)**	Centraliza o consumo nos créditos do Google AI Studio, oferece alta qualidade semântica em PT-BR e suporte a `768` dimensões (MRL) para eficiência no `pgvector`.
Embedding Alternativo	**`perplexity/pplx-embed-v1-0.6B` (OpenRouter)**	Documentado como alternativa de contingência ou redução extrema de custo (**US$ 0,004 / 1M tokens**) caso seja necessário desacoplar a vetorização dos créditos Google.
Observabilidade e Auditoria	**Pydantic Logfire + Tabela de Auditoria por Mensagem no Postgres**	**Logfire** (free tier) cobre traces técnicos, tokens e latência por etapa. O **PostgreSQL** persiste a auditoria de negócio vinculada a cada `message_id` (fontes, modelo, regras disparadas).

Domínio	Decisão Consolidada	Justificativa Técnica e Econômica
Proteção de Dados (PII)	**Pseudonimização de Dados Pessoais antes dos LLMs**	Dados pessoais e financeiros de produtores (`produtores.csv`, `financeiro.csv`) são mascarados na camada de ferramentas antes de qualquer envio a APIs externas.

3. Diretrizes e Pontos em Discussão para o System Design

Esta seção registra mecanismos de controle sugeridos e alternativas de modelagem para que o responsável pelo **System Design** avalie e defina a arquitetura definitiva em camadas da aplicação.

3.1 Sugestões de Pontos de Controle no Fluxo do Agente

1. **Ponto de Entrada (Guardrail, Cota e Cache no PostgreSQL):** Validação preliminar de escopo e tamanho de entrada sem chamadas LLM; consulta a cache leve em `UNLOGGED TABLE` do próprio PostgreSQL (para embeddings de perguntas repetidas e respostas puramente documentais/estáticas, invalidado por hash da carga de dados); controle rígido de orçamento de chamadas por turno.
2. **Acesso a Dados Somente Leitura (`READ-ONLY`):** As ferramentas do agente operam com conexão estritamente `READ-ONLY` sobre as tabelas operacionais normalizadas (CSV/JSON) e o índice vetorial (TXT no `pgvector`). Diferente de abordagens de *Text-to-SQL* livre, o agente invoca exclusivamente funções tipadas e parametrizadas.
3. **Tratamento de Atualização de Dados (Anti-Alucinação):** Consultas a dados mutáveis (estoque, financeiro, status sanitário) ocorrem em tempo real no banco a cada turno (zero cache de dados operacionais). Toda carga utiliza versionamento (`dataset_version` / SHA-256) com substituição transacional dos vetores antigos no `pgvector`.
4. **Ponto de Saída (Bloqueios Determinísticos e Auditoria):** Regras em Python validam bloqueios críticos (restrição sanitária `P017`, certificação vencida `P008`) antes da entrega, gravam a trilha de auditoria vinculada à mensagem e encerram o trace no Logfire.

3.2 Alternativas de Modelagem para Avaliação Arquitetural

Tópico de Design	Opção A	Opção B	Impacto Arquitetural
1. Gestão de Sessões no Chat	**Chat Único Ativo (Efêmero):** Mantém apenas a conversa atual na tela; iniciar nova conversa limpa o contexto ativo.	**Histórico Multi-Conversa:** Persiste múltiplas sessões no Postgres permitindo retomar atendimentos.	Define tabelas de sessão e UX no Streamlit. *(A auditoria por mensagem permanece salva em ambos os casos).*
2. Memória Ativa no LLM	**Janela Deslizante (Sliding Window):** Envia apenas os últimos N turnos (ex.: 3 perguntas e 3 respostas) sem payloads brutos de ferramentas.	**Resumo Progressivo + Janela Curta:** Compacta turnos antigos ao atingir limiar de tokens.	A Opção A evita contaminação de estoque antigo em novas perguntas e cabe com folga nos 32k tokens do Qwen3.

Tópico de Design	Opção A	Opção B	Impacto Arquitetural
3. Busca Documental (`.txt`)	**Busca Híbrida (Lexical/ID + `pgvector`):** Combina busca exata por códigos (`P008`, `2024-089`) com busca semântica.	**Apenas Busca Vetorial:** Depende exclusivamente da similaridade de cosseno dos embeddings.	A busca híbrida aumenta significativamente a precisão em códigos de lotes e laudos sanitários.
4. Efeitos Colaterais (Saídas)	**Somente Rascunho em Tela:** Exibe o JSON da triagem ou texto de notificação para revisão/cópia do operador.	**Persistência como Pendente:** Grava o rascunho em tabela de fila com status `PENDING_APPROVAL`.	Define se o agente expõe uma ferramenta isolada de escrita controlada com aprovação humana.

3.3 Referência Visual de Pipeline Determinístico (Modelo Conceitual)

O fluxograma abaixo ilustra um pipeline de referência baseado em **PydanticAI** (com guardrails de entrada, cache local, portão de cota, memória deslizante de sessão, acesso somente leitura e pós-processamento), servindo como modelo conceitual para o desenho final do *SaborDaTerraAI* (substituindo a etapa de SQL livre por ferramentas tipadas e `pgvector`):

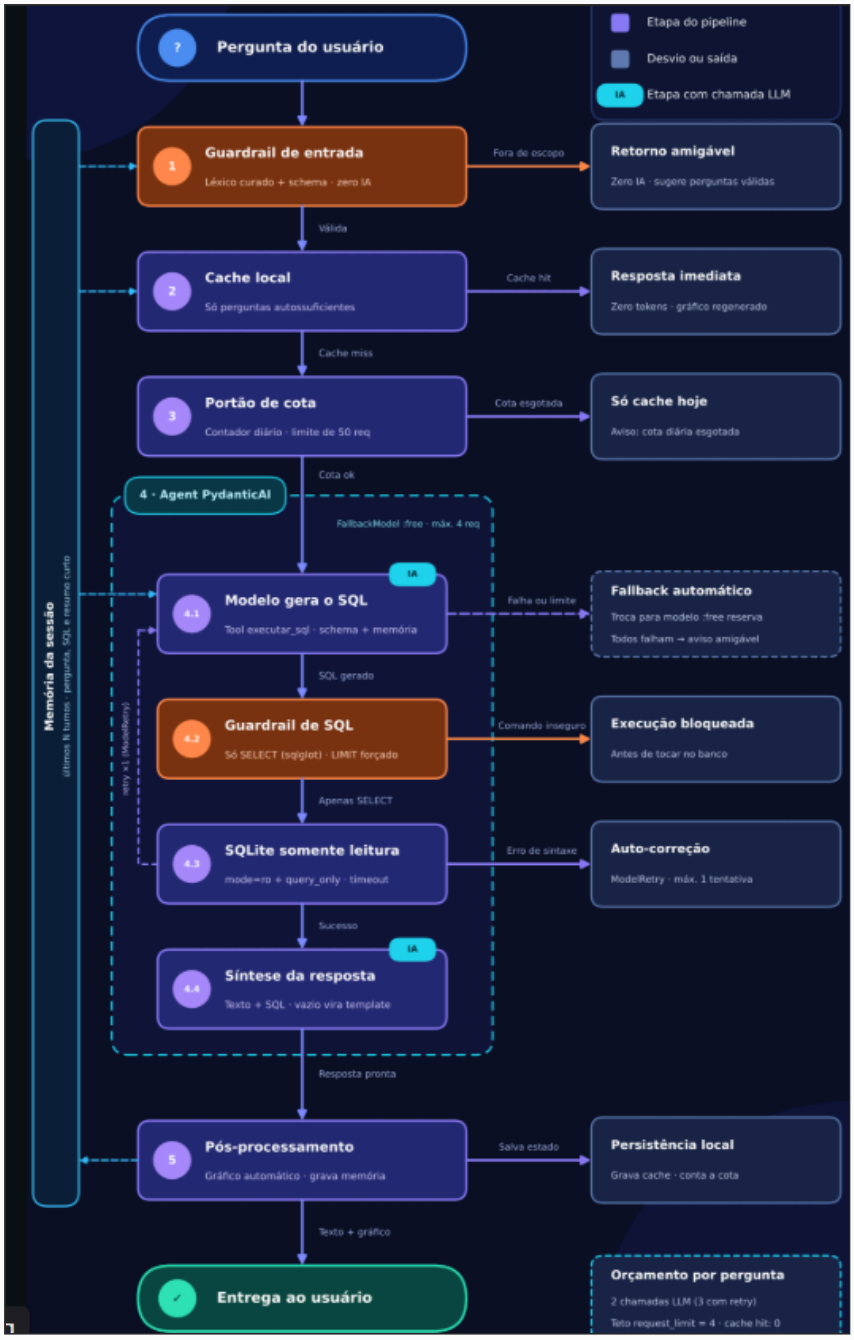


Figura 1 — Arquitetura de Referência do Pipeline: Isolamento determinístico em torno do núcleo PydanticAI, controlando orçamento de chamadas, memória deslizante dos últimos N turnos, acesso `READ-ONLY` ao banco e persistência de estado/auditoria no pós-processamento.

4. Evidências de Benchmark e Política de Failover

4.1 Resultados Medidos (API OpenRouter — 78 Chamadas Totais)

Os ensaios avaliaram 15 mensagens de triagem, 3 escolhas de ferramenta e 3 decisões baseadas em trechos documentais (execução única por caso, sem execução real de banco de dados):

Modelo via OpenRouter	Classe Correta	Esclarecimento	Ferramenta	Decisão com Fonte	Latência > 10s	Custo (21 calls)
Qwen3 32B	13/15	**13/15**	3/3	2/3 *(1 saída inválida)*	4/21	US$ 0,00226564
Mistral Small 3.2	10/15	10/15	3/3	2/3	**0/21**	**US$ 0,00088512**
DeepSeek V3.2 *(comparador)*	**14/15**	8/15	3/3	2/3	**0/21**	US$ 0,00269603

> **Leitura Técnica dos Benchmarks (Custo Total: US$ 0,00702325 em 78 chamadas):** Inclui US$ 0,00117647 da triagem inicial de 5 candidatos (onde `GLM 5.3 Flash` retornou respostas vazias e `Qwen3.5 Flash` retornou HTTP 400). Nenhum modelo do OpenRouter demonstrou confiabilidade suficiente para aplicar bloqueios críticos sozinho (Mistral e DeepSeek não sinalizaram proativamente a suspensão sanitária em `P017`, e Qwen3 excedeu 10s em 4 chamadas), validando a decisão de manter regras sanitárias e de certificação em código Python determinístico.

4.2 Política de Roteamento e Contingência

- **Cadeia de Execução:** `Gemini 3.5 Flash-Lite (Google AI Studio)` → `Qwen3 32B (OpenRouter)` → `Mistral Small 3.2 (OpenRouter)`.
- **Gatilhos de Fallback:** Timeout de rede, erros HTTP `5xx` ou `429` (Rate Limit), ou falha de validação de schema Pydantic.
- **Não-Gatilhos:** Ausência de dados no banco, ambiguidade na pergunta do usuário ou bloqueio de regra de negócio **nunca** disparam troca de modelo.
- **Orçamento Global de Tempo:** O teto de **10 segundos** aplica-se à requisição completa; se o modelo primário falhar próximo ao limite, a aplicação devolve falha segura sem acionar fallbacks tardios.

5. Análise Comparativa: Arquitetura Escolhida vs. Stack Nativa Google

Considerando a valorização institucional do ecossistema Google Cloud pela parceria UPE / Sauter, esta seção documenta os critérios técnicos e financeiros que fundamentam a escolha da arquitetura atual frente à adoção integral de serviços gerenciados de IA do Google (**Google ADK + Vertex AI + Gemini File Search**).

Dimensão	Arquitetura Proposta (PydanticAI + Cloud Run/SQL + AI Studio)	Caminho Nativo Google (Google ADK + Vertex AI + File Search)
Orquestração do Agente	**PydanticAI (Python):** Controle granular sobre schemas, regras determinísticas fora do prompt, mascaramento de PII e circuit breaker customizado entre provedores.	**Google ADK:** Framework oficial com sessões e ferramentas nativas, porém exigiria customização extra para orquestrar fallbacks externos via OpenRouter.
Aproveitamento de Créditos	**Gemini API (Google AI Studio):** Consome diretamente os créditos já disponíveis no AI Studio para geração e embeddings (`gemini-embedding-001`), sem custo extra de plataforma.	**Vertex AI / Agent Platform:** Créditos do Google AI Studio geralmente não são transferíveis para faturamento via Vertex AI, gerando cobrança direta na conta GCP.

Dimensão	Arquitetura Proposta (PydanticAI + Cloud Run/SQL + AI Studio)	Caminho Nativo Google (Google ADK + Vertex AI + File Search)
RAG e Dados Mistos	**PostgreSQL + `pgvector`:** Permite cruzar na mesma consulta/transação dados tabulares (CSV/JSON) e trechos vetoriais (TXT) com controle total de chunking e citações.	**Gemini File Search:** Serviço gerenciado que abstrai chunking e indexação (cobra indexação e tokens recuperados), mas não substitui consultas relacionais exatas.
Infraestrutura e Custo GCP	**Cloud Run (Scale-to-Zero) + Cloud SQL:** Pagamento apenas pelos segundos ativos no Cloud Run; instância Cloud SQL pode ser pausada fora das janelas de teste/apresentação.	**Cloud Run + Cloud SQL + Serviços Gerenciados Vertex:** Maior superfície de serviços ativos e maior custo fixo/variável caso recursos de busca gerenciada sejam habilitados.
Observabilidade	**Pydantic Logfire (Free Tier) + Auditoria SQL:** Custo zero para traces semânticos de LLM/tools e trilha de auditoria customizada no banco.	**Cloud Trace / Cloud Logging:** Integrado ao GCP, porém mais verboso para inspeção granular de prompts, schemas e chamadas de ferramentas.
Portabilidade (Lock-in)	**Alta:** O mesmo contêiner roda idêntico no Docker Compose local e no Google Cloud Run, sem acoplamento a APIs proprietárias de orquestração.	**Baixa/Média:** Maior dependência das abstrações de sessão, grounding e conectores específicos do ecossistema Google.

6. Contorno para o Desenho de System Design

Para orientar a elaboração do desenho arquitetural definitivo da aplicação (entrega obrigatória em **19/11/2026**), a matriz abaixo delimita o baseline já estabelecido e o escopo aberto à modelagem técnica do **System Design**:

Premissas Estabelecidas (Baseline Travado)	Escopo de Modelagem do System Design
• **Agente Único** em Python com **PydanticAI** e regras críticas determinísticas fora do prompt. • Empacotamento modular monolítico (**Streamlit + Agente**) em contêiner único Docker visando **Google Cloud Run**. • Instância única de **PostgreSQL + `pgvector`** (Cloud SQL), sem Redis ou mensageria externa. • Roteamento `gemini-3.5-flash-lite` → `Qwen3 32B` → `Mistral Small 3.2` dentro de SLA global de **≤ 10s**. • Vetorização exclusiva de documentos `.txt` via `gemini-embedding-001` (`768` dims; contingência Perplexity `US$ 0,004/1M`), mantendo CSV/JSON em SQL tipado. • Telemetria via **Pydantic Logfire** + auditoria persistida por mensagem no Postgres + mascaramento de PII antes dos LLMs. • Data de referência configurável (`30/10/2026`) e bloqueios obrigatórios para anomalias (`P008` , `P017`).	• **Arquitetura Interna em Camadas:** Organização dos módulos da aplicação (gateway de entrada, orquestração, serviços de domínio, repositórios e pós-processamento). • **Contratos e Ferramentas:** Definição formal dos schemas Pydantic de entrada/saída e assinaturas de cada ferramenta tipada (*Tools*). • **Modelagem Física de Dados (DDL):** Esquema das tabelas operacionais, índices vetoriais no `pgvector` , tabelas `UNLOGGED` de cache e tabela de auditoria. • **Política de Sessão e Memória:** Decisão final entre chat efêmero ou histórico persistido, tamanho exato da janela deslizante (N turnos) e parâmetros da busca híbrida. • **Pipeline de Ingestão:** Regras de normalização de unidades mistas (`quantidade_kg`) e controle de versão por hash SHA-256. • **Diagramas Arquiteturais:** Diagramas de componentes, implantação (GCP) e sequência (fluxo nominal e fluxo de contingência).